ANALYSIS AND OPTIMIZATION OF ENGINES FOR DYNAMICALLY TYPED LANGUAGES

Gem Dot
Universitat Politècnica de Catalunya
Barcelona, Spain

Alejandro Martínez
ARM
Cambridge, United Kingdom

Antonio González
Universitat Politècnica de Catalunya
Barcelona, Spain

***Abstract*— Dynamically typed programming languages have become very popular in the recent years. These languages ease the task of the programmer but introduce significant overheads since assumptions about the types of variables have to be constantly validated at run time. JavaScript is a widely used dynamically typed language that has gained significant popularity in recent years. In this paper, we provide a detailed analysis of the two main sources of overhead in the JavaScript execution. The first one is the runtime overhead needed for dynamic compilation and housekeeping activities (i.e. garbage collector, compilation, etc.). The second one is the additional checks and guards introduced by the dynamic nature of JavaScript. Then, we propose three new HW/SW optimizations that reduce this latter type of overhead. We show that these two types of overhead represent 35% and 25% respectively of the total execution time on average for a representative workload, and the proposed optimizations provide a 6% average speedup.**

Keywords—Dynamic typed languages; Dynamic Compilation; Computer Architecture

I. INTRODUCTION

Dynamically typed programming languages have gained a lot of adaptors and have become one of the most popular programming languages in the recent years [34]. JavaScript [8] is the most popular one; Python, PHP and Ruby are other commonly used languages.

These programming languages are executed by a virtual machine that translates/interpret the source code to the underlying machine language at run time. This approach is usually referred to as Just-in-Time compilation (JIT). The performance of the applications developed in these languages depends on two factors: the associated overheads and the quality of the generated code.

The main overheads are due to the time spent in dynamic compilation and interpretation, garbage collection and other housekeeping tasks. To reduce the overhead of these tasks, several techniques have been proposed, such as using staged compilation or a multithreaded runtime.

To improve the quality of the runtime code, modern JITs implement very advanced optimizations. However, they are limited by the fact that dynamically typed languages require that many checks are introduced in the code to verify speculative assumptions about data types.

In this work we use JavaScript as a vehicle to demonstrate the benefits of the proposed technique. In particular, we use the JavaScript engine from Google, which is known as V8 [9, 11, 15, 16]. Other vendors have alternative engines for JavaScript. Apple uses Nitro (previously known as SquirrelFish Extreme [26]), Mozilla uses SpiderMonkey [19] and Microsoft uses Chakra [30]. The proposed scheme can be applied to other dynamically typed languages and engines. Important efforts to improve the performance of these engines have been done in the last years [17, 18, 10, 14, 20, 2].

V8 compiles JavaScript to native machine code before executing it, which is dynamically profiled and subsequently optimized. Some of these dynamic optimization techniques are based in the Self-93 compiler [12, 5, 1, 13]. To understand the behavior of this dynamic compiler, the first part of this paper presents an analysis of the amount of executed instructions that are not part of the JavaScript application itself. These instructions can be considered as overhead produced by the V8 engine, which is a consequence of the characteristics of the JavaScript language along with the dynamic compilation nature of V8. This analysis is done for V8 targeting x86-64 processors.

In the second part of this paper, we use the insights gained from this study to improve certain parts of V8, concretely the overhead due to the dynamic nature of JavaScript. The optimizations that we propose are based on hw/sw codesigned techniques that incorporate new x86-64 instructions tailored for dynamic languages.

Previous studies with similar aims can be found at [24, 22, 3, 4, 6]. A further similar work is [20], which is focused on Nitro JavaScript compiler (used in Safari5 browser). It deals with quantifying the cost of type-checks on mobile processors and proposes an instruction set extension to address it. Although Nitro's design is comparable to V8 JavaScript engine, they differ in their approaches to dynamic inference and caching of user-defined types. However, these differences do not affect the study of type checking. Our work is different because we provide a more thorough analysis of JavaScript engine overheads, and because we propose more general optimizations, covering more parts of the JavaScript engine overhead.

Other works have also focused on analyzing the dynamic behavior of JavaScript programs [24] and provide a type system for JavaScript [22, 3, 4]. However, constructs of the language, such as the prototype system, are notoriously difficult to fit into a static typing discipline.

The main contributions of this paper are three:

- We quantify and analyze two important kinds of overhead in modern JavaScript execution.

1550-6533/15 $31.00 © 2015 IEEE
DOI 10.1109/SBAC-PAD.2015.20

IEEE computer society

Authorized licensed use limited to: Ryerson University Library. Downloaded on March 18,2026 at 06:06:52 UTC from IEEE Xplore. Restrictions apply.

- We perform an in-depth analysis of the overhead due to the dynamic aspects of JavaScript.
- Finally, we propose three HW/SW codesigned optimizations, that reduce this overhead and provide a 6% average speedup.

The remainder of this paper is organized as follows: Section 2 provides an overview of the most important aspects of V8 JavaScript engine. Section 3 presents a characterization of the execution of Octane, Kraken and SunSpider suites on the V8 JavaScript engine. In Section 4, three new HW/SW based optimizations that reduce the overhead of the JavaScript engine are presented. Section 5 concludes this paper.

II. THE V8 JAVASCRIPT ENGINE

V8 is a new JavaScript engine specifically designed for fast execution of large JavaScript applications. Depending on the nature of a specific program, its performance is normally better if it runs the same functions repeatedly, instead of running many different functions very few times each. This is because V8 focuses on optimizing hot functions (i.e. those functions that execute more often). To understand this, it is important to consider the V8 basic structure. V8 integrates two compilers, one that runs fast (i.e. has a light overhead) and produces code (Full-codegen) with practically no optimizations; and one that does not run as fast but generates optimized code (Crankshaft) [33].

A. Full-codegen compiler

The quick-and-simple compiler is known internally as the "Full-codegen" compiler. It takes as input the abstract syntax tree (AST) of a function, walks over the nodes in the AST, and emits calls to a macroassembler directly. Figure 1 outlines the flow of data at the compilation process. Full-codegen is a simple compiler that does not store local variables in registers; instead these are stored either on the stack or on the heap. All variables stored on the heap belong to a context object, which is associated to a function. When the value of a local variable is needed, the compiler emits a load to pull the value into a register [33].

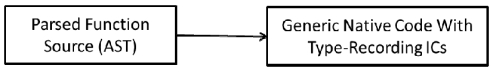

Fig. 1. V8 Full-codegen compiler.

Full-codegen compiler performs only one optimization, which is called Inline Caching (IC) [21]. This optimization has a twofold purpose: recording information concerning the types of objects and improving the performance of the system lookup routine used to disambiguate the type of objects when they are accessed.

For each call that comes from a new object property access, a system lookup routine has to access that object property. This routine performs a sequence of steps that determine the type of the object (because of the weakly typed nature of the language) and find the offset for that accessed property (Figure 2a). Since this process is quite costly, Inline Caching consists of caching the receiver type of the property when it is accessed first. Later, the subsequent accesses are substantially faster. Once the optimization is done, a method prologue checks whether the receiver type is the expected one (Figure 2b). Inline caching is used for assignments, unary and binary operations, function calls, and comparisons.

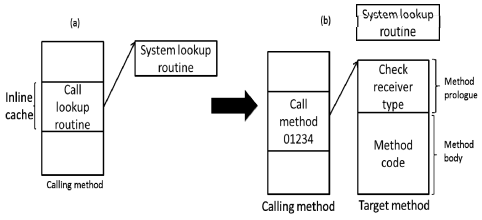

Fig. 2. Basic Inline Caching process.

The recorded information by the inline caches is also used by Crankshaft (the optimizing compiler) to perform more aggressive optimizations when functions are hot.

B. Crankshaft compiler

Once V8 has identified that a function is hot (by profiling) and has collected some type information through the inline caches, it tries to compile that function through the optimizing compiler (Crankshaft). As we can see in Figure 3, the process of Crankshaft is more sophisticated than Full-codegen [32].

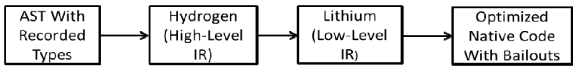

Fig. 3. V8 Crankshaft compiler.

Crankshaft first translates the JavaScript AST to a high-level static single-assignment Intermediate Representation (IR), which is called Hydrogen. In this part of the process, the compiler tries to apply high-level optimizations, such as Inlining. Then Hydrogen representation is translated to a machine-specific Low-level Intermediate Representation (LIR), which is called Lithium. This representation facilitates other machine-specific optimizations. Finally, Register Allocation and Code Generation are performed.

C. V8 hidden classes

Prototype-based programming is an object-oriented style in which classes are not present. Instead, the behavior reuse (i.e. the inheritance mechanism) is performed via reference of existing objects used as prototypes. In JavaScript all objects have a prototype property, which can be either another object or null, thereby forming a prototype chain. Therefore, when accessing a property of an object, the entire prototype chain is searched.

Hidden classes are unique entities introduced by V8 (they are not part of the JavaScript semantics) that represent object structures. In this regard, objects constructed in the same way by a function constructor share the same hidden class. Therefore, objects that share the same hidden class have the same type. When at runtime a function constructor creates an object for the first time, its hidden class is also created.

Due to the use of hidden classes, accessing a property of an object implies that some checking has to be performed.

Authorized licensed use limited to: Ryerson University Library. Downloaded on March 18,2026 at 06:06:52 UTC from IEEE Xplore. Restrictions apply.

This is in order to be sure that the type of the object is the expected one.

D. V8 execution phases

V8 is a dynamic compiler and therefore, program execution and code generation have to be efficiently synchronized, in order not to affect responsiveness. Moreover, JavaScript is a managed memory language, which needs a mechanism to reclaim memory that will no longer be used. Taking into account these characteristics, V8 execution can be broken down into the next phases:

- Non-optimizing compiler: V8 runtime executing Full-codegen compiler.
- Execution of non-optimized code: Execution of JavaScript application code produced by Full-codegen compiler.
- Optimizing compiler: V8 runtime executing Crankshaft compiler.
- Execution of optimized code: Execution of JavaScript application code produced by Crankshaft compiler.
- Garbage collector: V8 runtime mechanism that reclaims memory used by objects that are no longer required.
- Shared libraries: Execution of JavaScript shared libraries included in V8.
- Helpers: Execution of helper code to carry out some aspects of the JavaScript engine, such as built-ins corresponding to JavaScript construct calls.

As we will see later, the execution of optimized code is the most important part. Therefore, we will further analyze and quantify it.

E. V8 execution operations in optimized code

In the execution of optimized code there is a significant amount of instructions used for checking operations. However, these instructions do not belong to the application itself. In this section, we describe these operations. The specific checking operations are detailed in Table 1.

When the assumptions of these checking operations are not fulfilled, the optimized code is deoptimized. Moreover, there are other situations where the code can also be deoptimized and thus require also a check. Mainly, these other cases are encountered either in certain math operations (for instance, the assumption that a sum operation does not produce overflow) or in operations related to the object property accesses.

Besides checking operations, there are other instructions that do not belong to the application itself either. We have classified them as: Conversion instructions; and Tagging/Untagging instructions. Conversion instructions are necessary when number values need to have a different representation, because of subsequent uses. Tagging and Untagging instructions are used to box and unbox the number values.

TABLE I. V8 EXECUTION CHECK OPERATIONS CLASSIFICATION

Check Map	The first slot in each V8 object points to its class identifier (corresponding to its hidden class, which is its type). In this case, the type of a object is checked to be the same as that of another type, which has been seen before.
Check SMI	A register containing a boxed object can be of two types: either a SMI (small integer), which has its last bit cleared or a pointer, which has its last bit set. In this case, the last bit of a register is checked to know if it is a SMI.
Check Non-SMI	The opposite of check SMI.
Check Instance Type	It checks whether the type of a particular instance is the expected type.
Check Function	This is used to check whether an inlined function is the expected one.
Check Prototype Map	There are accessible properties of an object that reside in its prototype chain. Therefore, when a function call belonging to a prototype object is inlined, it is necessary to introduce a type check of that prototype.
Check Map Value	It is like a check map, although only in case of enumerable objects in a for-in statement
Check Bounds	After obtaining the total length of the array, it is compared to the current index, in order to check that the accessed position of the array is not out of bounds.
Check Stack	Inside a loop, the stack limit is checked to see if it has been reset, in order to know whether an interruption has been produced.

III. PERFORMANCE ANALYSIS

In this section, we characterize the different execution phases of the V8 engine in steady state. After this, we focus on a more detailed analysis of the execution of optimized code, since it represents the most important phase in steady state.

We use three suites to analyze the V8 dynamic compiler: Octane [11], Kraken [35] and SunSpider [27]. SunSpider is a JavaScript benchmark suite for measuring the performance of JavaScript engines with more than a dozen tests, each concentrating on a different part of the JavaScript language. SunSpider focuses on computation (does not include HTML, CSS, networking, etc.).

A. V8 execution phases

Figure 4 shows the contribution of the different phases to the total execution time of the V8 engine. Results were obtained through the V8's built-in sampling profiler [11]. It is important to note that most of the time is consumed executing optimized code and only a very small percentage of time is dedicated to the execution of non-optimized code. In addition, compilation time represents around 24% of the total execution for the Octane and Kraken suites and 14% for SunSpider. In the Octane suite, the garbage collector is a very important contributor for *splay* and *earley-boyer* benchmarks, which make an important use of heap memory. Nevertheless, in this paper, we do not focus in the garbage collector because this is a widely covered topic elsewhere. Note also that *code-load, gbemu, pdfjs, regexp, audio-oscillator, imaging-darkroom, json-parse, json-stringify, date-format, 3d-morph, math-partial, regexp-dna, string-unpack, string-validate* benchmarks make a poor use of optimized code. These benchmarks have little code reuse and, therefore, only a small fraction of the code is optimized.

Authorized licensed use limited to: Ryerson University Library. Downloaded on March 18,2026 at 06:06:52 UTC from IEEE Xplore. Restrictions apply.

B. *Optimized code execution*

Next, we focus on the steady state, to minimize the execution of non-optimized code. We achieve steady state by executing the benchmark ten times and taking statistics from the tenth iteration. We have run each benchmark twice. In the first run, we have counted the total number of x86-64 executed instructions by the V8 runtime using pin [7]. In the second run, we introduced additional assembler code (i.e. dynamic counters) to the V8 runtime in order to gather some statistics of interest (e.g. number of times a given check is performed). Results are presented as percentages with respect to the original unmodified code.

Figure 5 shows the dynamic instruction count broken down into several categories. Allocate Objects is the amount of executed instructions to create new objects in the heap. Rest of code means the non-optimized code, the garbage collector, compilers, etc. In this Figure, we can observe a similar amount of total overhead for the three suites, with a similar and significant percentage of Checks in all cases. In addition, Tags and Untags have an important contribution only for a few benchmarks of SunSpider and *raytrace* benchmark, from Octane. Furthermore, the benchmarks that have a low ratio of optimized code, also present a small overhead, except *Math-partial-sums*, *date-format-tofte* and *string-unpack-code*, from SunSpider.

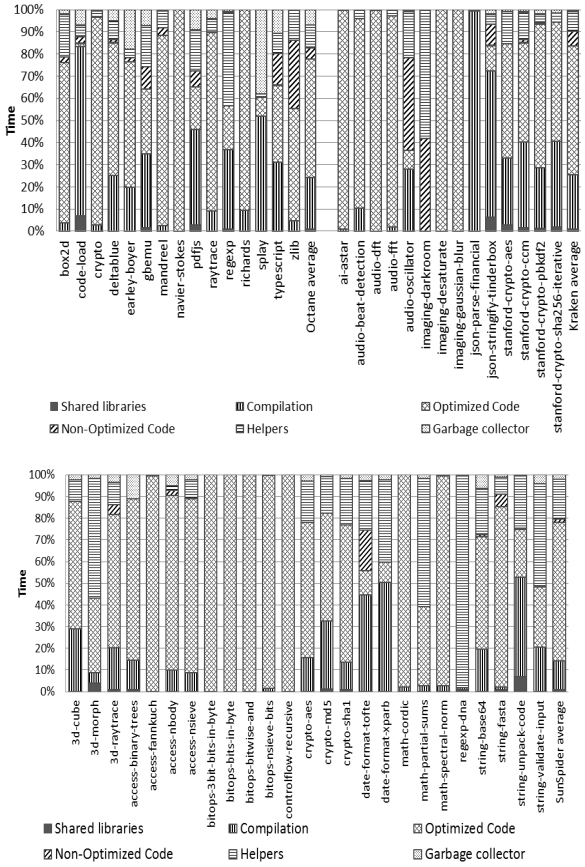

Fig. 4. V8 engine execution phases.

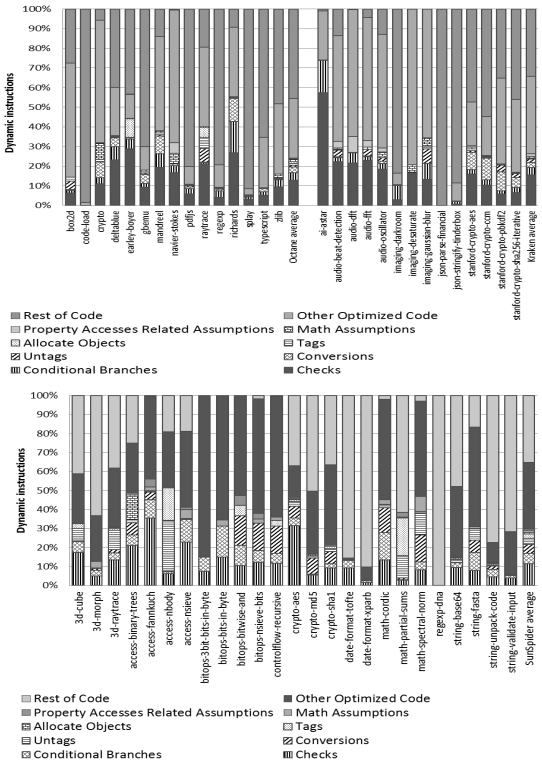

Fig. 5. Breakdown of dynamic instructions.

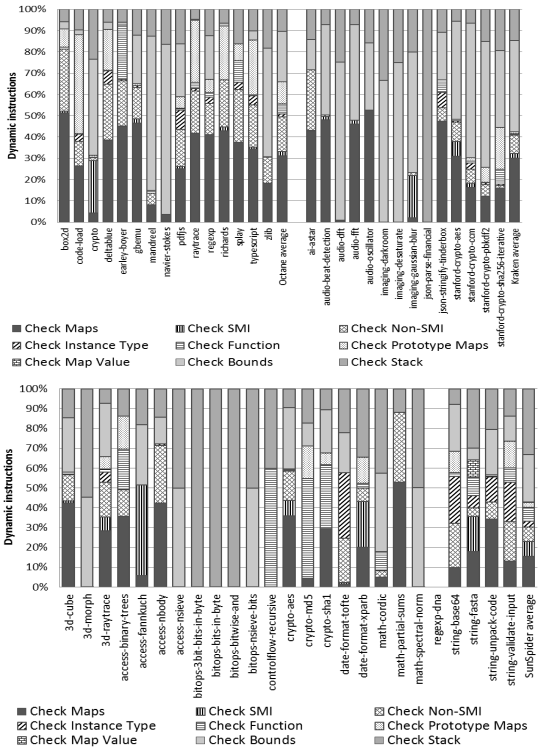

Fig. 6. Breakdown of Check operations.

Authorized licensed use limited to: Ryerson University Library. Downloaded on March 18,2026 at 06:06:52 UTC from IEEE Xplore. Restrictions apply.

Figure 6 shows the dynamic instruction breakdown for Checks. *Check Map*, *Check Stack,* and *Check Bounds* are the most important checks for SunSpider and Kraken, whereas *Check Maps*, *Check-Non-SMI*, *Check Bounds,* and *Check Prototype Maps* are the most important ones for Octane. *3d-morph*, *Access-nsieve*, *bitops-3bit-bits-in-byte, bitops-bits-in-byte, bitops-bitwise-and, bitops-nsieve-bits* and *math-spectral-norm* benchmarks, from SunSpider, *navier-stokes and mandrel* benchmarks from Octane, and *audio-dft*, *imaging-darkroom, imaging-desaturate* and *json-parser-financial*, from Kraken, only have *Check Bounds* and *Check Stack* operations as overhead. These are loop-intensive benchmarks, and V8 generates a *Check Stack* operation for every loop iteration to know if an external exception has happened. Finally, benchmarks with a high percentage of both *Check Maps* and *Check Non-SMI* operations also are very common.

In this section, we have quantified the contribution of checks, conversions and other operations in the execution of JavaScript applications on the V8 JavaScript engine. These operations are not part of the JavaScript application. Instead, these additional instructions can be considered as overhead produced by the V8 engine, as a consequence of the dynamic characteristics of the language. As we have seen, the most frequent operations are those that check assumptions. However, Conversion and Conditional branch operations also have a significant contribution.

IV. HW/SW CODESIGNED BASED OPTIMIZATIONS

There is a great opportunity to improve the performance of JavaScript platforms by reducing the overheads quantified in the previous section. In this regard, we propose three optimizations that are supported by the introduction of new hardware and the necessary software changes. In this work we assume an x86-64 processor, so we introduce several new x86-64 instructions to support these optimizations.

The proposed optimizations have been simulated at instruction level and also at micro-architectural level through a cycle-level simulator. We describe these optimizations below.

A. Deoptimization Bailout and Check Stack optimization

Every time the optimized code is checked to potentially deoptimize it, two instructions are used. The first one is an instruction that changes a flag. This instruction is usually a *test* or a *cmp* instruction. The second one is a conditional branch instruction that jumps to a Deoptimization Bailout depending on the flag set by the previous instruction. We have observed that optimized code is rarely deoptimized.

The idea behind Deoptimization Bailout optimization is replacing these two instructions by a new one (Figure 7). This new instruction performs an ALU operation, checks a flag specified in the instruction and raises an exception in case that the check succeeds. The key point is that in the vast majority of cases (almost 100%) the code is not deoptimized. Therefore, whenever the code is not deoptimized, less dynamic instructions are executed and branch prediction is not needed.

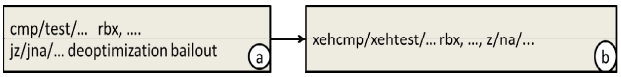

Fig. 7. Deoptimization pattern improvement.

If the code has to be deoptimized, a hardware exception is thrown. This exception is intercepted by a handler in the V8 runtime that executes a special routine, which finds the action to do according to the current program counter. This action is a jump to an address that targets a specific deoptimization bailout. The overhead is negligible compared with the deoptimization routine itself.

We have introduced eleven new x86-64 instructions to allow the implementation of this optimization, which are: *xehtest*; *xehxor*; *xehadd*; *xehneg*; *xehcmp*; *xehucomis*; *xehand*; *xehimull*; *xehor*; *xehsub*; and *xehassert*. They perform a math operation (test, xor, add, etc.) and check the resulting value of a specific flag. Moreover, each new instruction is able to throw a HW exception according to the encoded specific flag. Note that *xehassert* instruction checks a specific flag generated by a previous instruction instead of the instruction itself.

We can use the same exception mechanism to optimize the *Check Stack* pattern. This check uses the same routine as the deoptimization bailout to find the action to do. However, in this case, the action is a call that interrupts the program because another external exception has taken place. Figure 8 represents this optimization at instruction level.

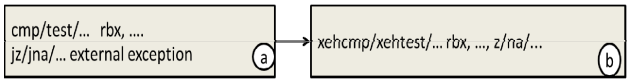

Fig. 8. Check Stack improvement.

Figure 9 outlines the required hardware support for these two optimizations. A mux is added in the execute stage to select the flag to check that is compared with the expected value. The result of this comparison is saved in the exception bit. If this bit is set, a HW exception will be thrown at the commit stage. Note that the Hardware solution for this optimization does not involve significant extra computation and, in addition, the critical path is not affected, because the result is not needed until the commit stage.

These optimizations can also be applied to other common JavaScript engines. For instance, IonMonkey [29] and Nitro [20] introduce a guard just before unboxing an object, in order to ensure that the specific type of the object is the expected one. This guard also consists of a comparison and a subsequent conditional jump to a bailout that can be optimized in the same way as shown for V8. These guards also are used for other type of assumptions (e. g. arithmetic assumptions).

Authorized licensed use limited to: Ryerson University Library. Downloaded on March 18,2026 at 06:06:52 UTC from IEEE Xplore. Restrictions apply.

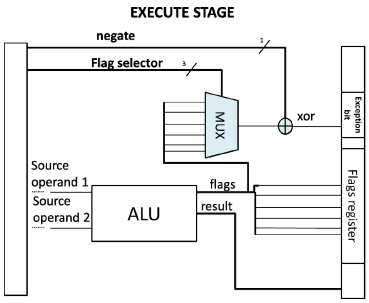

Fig. 9. Block diagram for the Deoptimization Bailout and Check Stack optimization.

B. Tagged-to-Integer optimization

Tagged-To-Integer conversion instructions are necessary when number values need to have a different representation, because of subsequent uses. These conversions represent 3% of the dynamic instructions for SunSpider on average, but it can reach up to 15% for some benchmarks.

Figure 10a presents the code for *Tagged To Integer* conversion after applying the optimizations presented in the previous section. We propose to join the *xehtest* and shift instructions in a new single one, which is called *xehtestshift* (Figure 10b). This new instruction shifts the value of the register and if the value is not a SMI, an exception is raised to a deoptimization code.

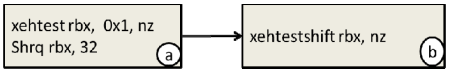

Fig. 10. Tagged-to-Integer pattern improvement.

Figure 11 presents a block diagram of the hardware required to support the xehtestshift instruction, which is an extension of the scheme of Figure 9. Note that another mux is introduced to select between the flag indicated by the previous optimization and the least significant bit, which indicates whether the value is a SMI. The selected bit is compared to the expected value and the result of this comparison is saved in the exception bit. If this bit is set, then a HW exception will be thrown at the commit stage.

Both Nitro and IonMonkey encode the integer tag into the most significant part of the 64 bit register. This optimization could be easily adapted for them.

C. Check Non-SMI and Check Map pattern optimization

Our analysis revealed that there are sequences of checks that are very frequent. The most repeated pattern is a *Check Non-SMI* followed by a *Check Map*, so in this section we present an optimization for it.

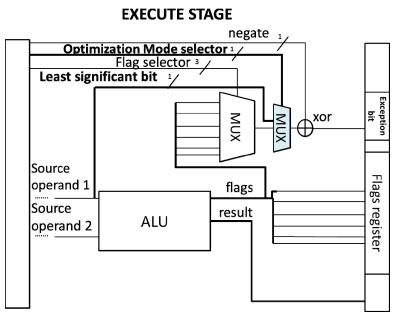

Fig. 11. Block diagram for the tagged-to-integer optimization.

Figure 12a presents the new code for the *Check Non-SMI* and *Check Map* after applying the optimization presented in section IV.A. We propose to join the *xehtest* and *xehcmp* instructions into a new one (Figure 12b), which is called *xehtestcmp*. This instruction checks whether the "*this"* pointer does not contain a SMI and whether it points to an object whose type is the expected one. If this is not the case, an exception that jumps to deoptimization code is produced. In this way, the number of dynamic instructions for this pattern is reduced to two.

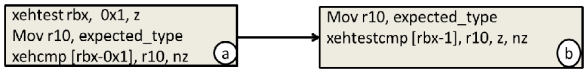

Fig. 12. Check non-SMI and check map pattern improvement.

As Both Nitro and IonMonkey encode the integer tag in the same way that the other object types, they do not need to use this optimization. They simply need to perform a single check for the expected type of the object.

D. Results

Below we present the performance of the above optimizations using three benchmark suites: Octane, Kraken and SunSpider. We use a timing simulator with a micro-architectural configuration closely matching a Nehalem core.

1) Dynamic instruction count improvements.

We executed every benchmark once in steady state and obtained the number of dynamic instructions using the Pin tool. Then, for each optimization, we obtained the reduction in instruction count through instrumentalization of the V8 engine (i.e. additional assembler code to the V8 runtime in order to identify the optimized operations). Figure 13 shows the results for Octane, Kraken and SunSpider suites.

All three suites get an important improvement from the Deoptimization Bailout optimization. Check Non-SMI and Check Map optimization improve significantly *ai-astar* and *crypto-aes* benchmarks, whereas Tagged-To-Integer optimization has an important impact for *Richards*, *controlflow-recursive* and *bitops-bitwise-and* benchmarks. These results include the cost of failing when the assumptions are not correct.

Authorized licensed use limited to: Ryerson University Library. Downloaded on March 18,2026 at 06:06:52 UTC from IEEE Xplore. Restrictions apply.

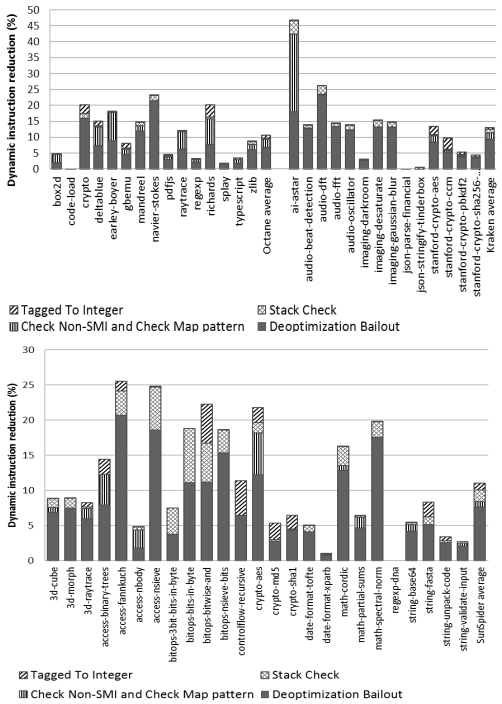

Fig. 13. Improvement in dynamic instructions.

2) Cycle count improvements

To measure cycle count improvements, we have simulated a single execution of every benchmark in steady state using the Sniper simulator [25]. For this purpose, each benchmark was previously executed 9 times for the warm up phase. Table 2 shows the micro-architectural features of the simulated core, which resemble those of a Nehalem core [28]. The optimizations can be identified and quantified independently because the Sniper is a causal simulator and the optimizations are decoupled among themselves. Figure 14 presents the results for Octane, Kraken and SunSpider suites. Note that Stack Check and Deoptimization Bailout optimizations are lumped together into the same column.

Regarding the Octane suite, Deoptimization Bailouts and Stack Check optimizations are the most effective for all applications, and improve this suite by 4% on average. *Crypto* and *Richards* benchmarks are the ones that benefit the most from Tagged-To-Integer optimizations, since these are integer benchmarks. Check Non-SMI and Check-Map Map pattern optimization improves *richards, raytrace*, *earley boyer, deltablue* and *box2d* between 1% and 2%. The other benchmarks are not improved because they do not perform many object property accesses. Some benchmarks present a 0% improvement. The main reason is that they execute very little optimized code and therefore, the overheads targeted by our optimizations are minimal.

Regarding the SunSpider suite, *Bitops-bitwise* and *crypto-md5* benchmarks benefit a lot from Tagged-To-integer optimization, because they are integer applications. Check Non-SMI and Check-Map pattern optimization results in small benefits. The rest of the improvement for this suite is mostly

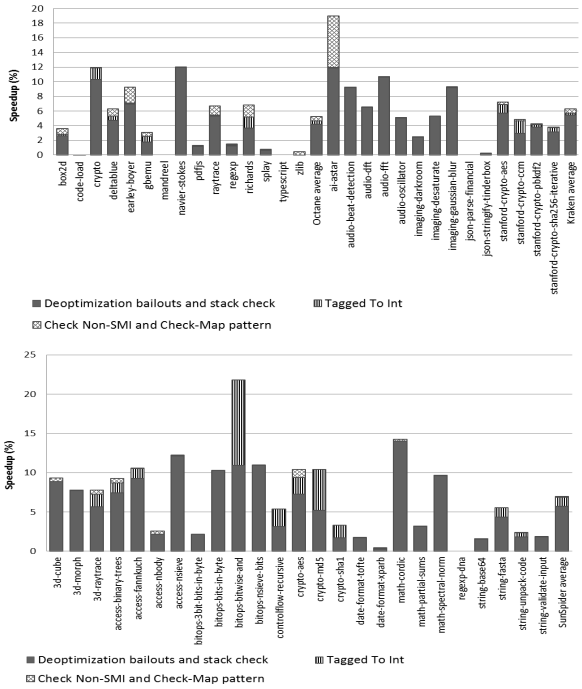

Fig. 14. Improvement in dynamic instructions.

TABLE II. SIMULATED MICRO-ARCHITECTURE CONFIGURATION.

Dispatch width	4
Window size	128
Outstanding load/stores	10
Itlb	128 KB, 4-way
Dtlb	256 KB, 4-way
Il1 cache	32 KB, 4-way
Dl1 cache	32 KB, 8-way
L2 cache	256 KB, 8-way
L3 cache	4096 KB, 16-way

due to the Deoptimization Bailout and Stack Check optimization.

Finally, in the Kraken suite**,** *ai-astar* benchmark is the only one that benefits from Check Non-SMI and Check-Map optimization. Cryptographic benchmarks are the only ones that benefit from Tagged-To-integer optimization. Again, the main improvement for this suite is mostly due to the Deoptimization Bailout and Stack Check optimizations.

Overall, the proposed optimizations reduce the number of dynamic instructions by 10.5%, 12.5% and 11% for Octane, Kraken and SunSpider suites respectively, and the cycle count by 5.2%, 6.2% and 7%.

V. CONCLUSIONS

JavaScript is a widely used dynamic language and its popularity is expected to increase in the future. In this paper, we have provided a quantitative analysis of the most important aspects of the V8 JavaScript engine, which is an open source and widely used JavaScript engine developed by Google.

We have evaluated the execution overhead, classifying it into two main categories. The first one is the code that V8

Authorized licensed use limited to: Ryerson University Library. Downloaded on March 18,2026 at 06:06:52 UTC from IEEE Xplore. Restrictions apply.

runtime executes for different housekeeping activities (i.e. garbage collector, compilation, etc.). The second one is due to the additional checks and guards introduced by the dynamic nature of JavaScript. Finally, guided by this study, we have provided three HW/SW based optimizations, which reduce this latter overhead.

The study shows that the overheads of JavaScript engines are important, and they represent around 60% (25% additional checks and guards, and 35% compilation, garbage collector and helpers) of the total execution time in steady state. The proposed optimizations target some of the most important sources of overhead of the first category. They are based on a hybrid HW/SW approach that requires the introduction of some new machine instructions, some additional hardware support and some changes in the code generated by the dynamic compiler. We have shown that these optimizations result in a 6% average speedup for representative benchmarks. In this paper, we have focused on V8 JavaScript engine. However, this analysis and optimizations can be extended to other JavaScript engines and other dynamic languages.

VI. Acknowledgments

This work has been partially supported by the Spanish Ministry of Economy and Competitiveness under grants TIN2010-18368 and TIN2013-44375-R and the Spanish Ministry of Education, Culture and Sport under grant FPU12/05670.

References

[1] Chambers, C., and Ungar, D. 1989. Customization:optimizing compiler technology for Self, a dynamically-typed object-orientied programming language. In Proceedings of the SIGPLAN'89 Conference on Programming Language Design and Implementation. 146-160.

[2] A. Gal, B. Eich, M. Shaver, D. Anderson, D. Mandelin, M. R. Haghighat, B. Kaplan, G. Hoare, B. Zbarsky, J. Orendorff, J. Ruderman, E. W. Smith, R. Reitmaier, M. Bebenita, M. Chang, and M. Franz. Trace-based Just-in-Time Type Specialization for Dynamic Languages. In Conference on Programming language Design and Implementation (PLDI), 2009.

[3] Christopher Anderson. Type Inference for JavaScript. PhD thesis, Department of Computing, Imperial College London, March 2006.

[4] Christopher Anderson and Paola Giannini. Type checking for JavaScript. Electr. Notes Theor. Comput. Sci., 138(2), 2005.

[5] David Ungar and Randall B. Smith. Self: The power of simplicity. In Proceedings OOPSLA '87, ACM SIGPLAN Notices, volume 22, pages 227–242, December 1987. 269

[6] Devesh Tiwari and Yan Solihin, Architectural Characterization and Similarity Analysis of Sunspider and Google's V8 Javascript Benchmarks. In IEEE International Symposium on Performance Analysis of Systems & Software, 2012.

[7] Drevor, Tevi, Pin : Intel's Dynamic Binary Instrumentation Engine, Pin Tutorial. Intel Corporation, 2010.

[8] ECMA. ECMAScript Language Specification – Fifth Edition. http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-262.pdf.

[9] Florian Schneider, High performance JavaScript with V8, 2012. http://cs.au.dk/~jmi/VM/IC-V8.pdf

[10] E. Fortuna, O. Anderson, L. Ceze, and S. Eggers. A limit study of javascript parallelism. In Proceedings of IISWC, pages 1–10, 2010.

[11] Google. V8 JavaScript engine, 2009. http://code.google.com/p/V8/.

[12] U. Holzle, Adaptive Optimization for Self: Reconciling High Performance with Exploratory Programming, PhD dissertation, Stanford Univ., Stanford, Calif., 1994.

[13] U. Holzle and D. Ungar. Optimizing Dynamically-dispatched Calls with Run-time Type Feedback. In Conference on Programming language Design and Implementation (PLDI), 1994.

[14] Jan Kasper Martinsen, Håkan Grahn, Anders Isberg. Using Speculation to Enhance JavaScript Performance in Web Applications.

[15] Mads Sig Ager, V8 internals,2009. https://dl.google.com/io/2009/pres/W_1230_V8BuildingaHighPerformanceJavaScriptEngine.pdf

[16] Mads Sig Ager, The V8 JavasCript engine. http://websrv0a.sdu.dk/ups/SCM/slides/lecture_03_mads_ager.pdf

[17] J. Martinsen and H. Grahn. An alternative optimization technique for javascript engines. In Proceedings of MCC, pages 155–160, 2010.

[18] M. Mehrara, P. Hsu, M. Samadi, and S. Mahlke. Dynamic parallelization of javascript applications using an ultra-lightweight speculation mechanism. In Proceedings of High Pergormance Computer Architecture, pages 87–98, 2011.

[19] Mozilla. Spidermonkey javascript engine. https://developer.mozilla.org/en-US/docs/Mozilla/Projects/SpiderMonkey

[20] Owen Anderson, Emily Fortuna, Luis Ceze, Susan Eggers. Checked Load: Architectural Support for JavaScript Type-Checking on Mobile Processors. Computer Science and Engineering, University of Washington. HPCA, 2011.

[21] L. Peter Deutsch and Alan Schiffman, Efficient Implementation of the Smalltalk-80 System. Proceedings of the 11th Symposium on the Principles of Programming Languages, Salt Lake City, UT, 1984.

[22] Peter Thiemann. Towards a type system for analyzing JavaScript programs. In European Symposium on Programming (ESOP), 2005.

[23] Pop, Sebastian (2006). The SSA Representation Framework: Semantics, Analyses and GCC Implementation.

[24] G. Richards, S. Lebresne, B. Burg, and J. Vitek. An Analysis of the Dynamic Behavior of JavaScript Programs. In Conference on Programming language Design and Implementation (PLDI), 2010.

[25] Trevor E. Carlson, Wim Heirman, and Lieven Eeckhout. Sniper: Exploring the level of abstraction for scalable and accurate parallel multi-core simulations. In International Conference for High Performance Computing, Networking, Storage and Analysis (SC), November 2011.

[26] WebKit. Introducing SquirrelFish Extreme. http://webkit.org/blog/214/introducing-squirrelfish-xtreme/, 2008.

[27] WebKit. SunSpider JavaScript Benchmark. http://webkit.org/perf/SunSpider-0.9/SunSpider.html , 2008.

[28] White Paper: Intel® Next Generation Microarchitecture (Nehalem), 2008.http://www.intel.com/pressroom/archive/reference/whitepaper_nehalem.pdf.

[29] Igor Rafael de Assis Costa, Henrique Nazaré Santos, Péricles Rafael Alves, Fernando Magno Quintão Pereira. Just-in-Time value specialization. Department of Computer Science, Federal University of Minas Gerais (UFMG), Brazil. In Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO).

[30] Chakra's Technical review. http://blogs.msdn.com/b/ie/archive/2014/10/09/announcing-key-advances-to-javascript-performance-in-windows-10-technical-preview.aspx

[31] Andy Wingo. v8: a tale of two compilers.http://wingolog.org/archives/2011/07/05/v8-a-tale-of-two-compilers , 2011.

[32] Andy Wingo. a closer look at crankshaft,v8's optimizing compiler. http://wingolog.org/archives/2011/08/02/a-closer-look-at-crankshaft-v8s-optimizing-compiler,2011.

[33] Andy Wingo. inside full-codegen, v8's baseline compiler. http://wingolog.org/archives/2013/04/18/inside-full-codegen-v8s-baseline-compiler

[34] Tiobe programming community Index. http://www.tiobe.com/index.php/content/paperinfo/tpci/index.html

[35] Mozilla. Kraken. https://krakenbenchmark.mozilla.org, 2013.

Authorized licensed use limited to: Ryerson University Library. Downloaded on March 18,2026 at 06:06:52 UTC from IEEE Xplore. Restrictions apply.